

Analisi di Complementarità: Matrice di Rotazione e Angoli ZYX

Di seguito viene analizzata la complementarità matematica tra la funzione di generazione della matrice di rotazione e la funzione di estrazione degli angoli.

1. Funzioni Python di Riferimento

Queste sono le due funzioni implementate per eseguire la conversione diretta e inversa:

```
import numpy as np

def create_rot_matrix_zyx(r):
    """
    Genera la matrice di rotazione a partire dalle rotazioni intrinseche ordinate
    """
    rx, ry, rz = np.radians([r[0], r[1], r[2]])

    # Matrici di rotazione elementari
    R_x = np.array([
        [1, 0, 0],
        [0, np.cos(rx), -np.sin(rx)],
        [0, np.sin(rx), np.cos(rx)]
    ])

    R_y = np.array([
        [np.cos(ry), 0, np.sin(ry)],
        [0, 1, 0],
        [-np.sin(ry), 0, np.cos(ry)]
    ])

    R_z = np.array([
        [np.cos(rz), -np.sin(rz), 0],
        [np.sin(rz), np.cos(rz), 0],
        [0, 0, 1]
    ])

    # Rotazione totale (Convenzione Z-Y-X Intrinseca)
    # P_G = R_z @ R_y @ R_x @ P_L + T
    R = R_z @ R_y @ R_x
    return R

def rot_matrix_to_angles_zyx(R):
    """
    Estrae gli angoli di Cardano (rx, ry, rz) in gradi da una matrice di rotazione
    L'ordine atteso è lo stesso di create_rot_matrix_zyx (ZYX intrinseco o XYZ es
    """
    # Calcoliamo la radice per verificare la presenza di singolarità (Gimbal Lock)
    sy = np.sqrt(R[0, 0]**2 + R[1, 0]**2)
    singular = sy < 1e-6

    if not singular:
        rx = np.arctan2(R[2, 1], R[2, 2])
        ry = np.arctan2(-R[2, 0], sy)
        rz = np.arctan2(R[1, 0], R[0, 0])
    else:
        # In caso di Gimbal Lock (ry = +/- 90 gradi)
```

```

rx = np.arctan2(-R[1, 2], R[1, 1])
ry = np.arctan2(-R[2, 0], sy)
rz = 0.0

r = np.array(np.degrees([rx, ry, rz]))
return r

```

Le due funzioni sono **matematicamente complementari**. Sotto specifiche condizioni standard, l'operazione sequenziale:

$$\mathbf{r} \xrightarrow{\text{Generazione}} R \xrightarrow{\text{Estrazione}} \mathbf{r}'$$

restituisce esattamente gli stessi angoli di partenza ($\mathbf{r}' \approx \mathbf{r}$), dove $\mathbf{r}$ rappresenta il vettore degli angoli $[r_x, r_y, r_z]$.

2. Chiarimento sulle Convenzioni: Intrinseco vs Estrinseco

Esiste un teorema fondamentale in cinematica 3D (Teorema di Dualità) che lega le rotazioni ad assi mobili (intrinseche) e ad assi fissi (estrinseche):

Una sequenza di rotazioni intrinseche eseguita in un certo ordine equivale esattamente alla stessa sequenza di rotazioni estrinseche eseguita in ordine inverso.

Nel codice della funzione `create_rot_matrix_zyx` esegui il prodotto:

$$R = R_z(r_z) \cdot R_y(r_y) \cdot R_x(r_x)$$

Questo prodotto rappresenta matematicamente due cose identiche:

1. **Convenzione ZYX Intrinseca (Assi Mobili):** Si ruota prima attorno a Z di r_z , poi attorno al nuovo asse Y' di r_y , infine attorno al nuovo asse X'' di r_x . Nelle rotazioni intrinseche, le matrici si moltiplicano da sinistra a destra nell'ordine in cui avvengono le rotazioni ($R_z \cdot R_y \cdot R_x$).
2. **Convenzione XYZ Estrinseca (Assi Fissi / Globali):** Si ruota prima attorno all'asse globale X di r_x , poi attorno all'asse globale Y di r_y , e infine attorno all'asse globale Z di r_z . Nelle rotazioni estrinseche, l'ordine di moltiplicazione è invertito rispetto all'ordine di esecuzione ($R_z \cdot R_y \cdot R_x$).

Pertanto, l'ordine è formalmente ZYX intrinseco (o XYZ estrinseco).

3. Corrispondenza Matematica

Sviluppando analiticamente il prodotto delle tre matrici elementari

$R = R_z(r_z) \cdot R_y(r_y) \cdot R_x(r_x)$, la matrice risultante è:

$$R = \begin{bmatrix} \cos(r_z) \cos(r_y) & \cos(r_z) \sin(r_y) \sin(r_x) - \sin(r_z) \cos(r_x) & \cos(r_z) \sin(r_y) \cos(r_x) \\ \sin(r_z) \cos(r_y) & \sin(r_z) \sin(r_y) \sin(r_x) + \cos(r_z) \cos(r_x) & \sin(r_z) \sin(r_y) \cos(r_x) \\ -\sin(r_y) & \cos(r_y) \sin(r_x) & \cos(r_y) \cos(r_x) \end{bmatrix}$$

La funzione di estrazione `rot_matrix_to_angles_zyx` esegue il percorso inverso sfruttando questa struttura di indici (dove $R[i, j]$ indica l'elemento alla riga i e colonna j):

- $R[2, 0] = -\sin(r_y)$
- $R[2, 1] = \cos(r_y) \sin(r_x)$
- $R[2, 2] = \cos(r_y) \cos(r_x)$
- $R[1, 0] = \sin(r_z) \cos(r_y)$
- $R[0, 0] = \cos(r_z) \cos(r_y)$

Calcolando la norma di $R[0, 0]$ e $R[1, 0]$, la funzione isola il termine $|\cos(r_y)|$ (la variabile `sy` nel codice):

$$sy = \sqrt{R[0, 0]^2 + R[1, 0]^2} = \sqrt{\cos^2(r_y)(\cos^2(r_z) + \sin^2(r_z))} = |\cos(r_y)|$$

Quando la matrice non è in una condizione di singolarità, l'estrazione degli angoli avviene così:

- $r_x = \text{atan2}(R[2, 1], R[2, 2])$
- $r_y = \text{atan2}(-R[2, 0], sy)$
- $r_z = \text{atan2}(R[1, 0], R[0, 0])$

Nota su $\text{atan2}(y, x)$

La funzione $\text{atan2}(y, x)$ (arcotangente a due argomenti) è fondamentale in questo processo di estrazione. A differenza della classica $\text{atan}(y/x)$, essa accetta separatamente i valori di y (numeratore) e x (denominatore) mantenendone i segni originali. Ciò consente di determinare in modo univoco l'angolo corretto in tutti e quattro i quadranti dell'intervallo $[-\pi, \pi]$ (ovvero $[-180^\circ, 180^\circ]$) ed evita l'indeterminazione o l'errore matematico dovuto alla divisione per zero quando $x = 0$.

4. Le Eccezioni alla Complementarità Diretta

Anche se la matrice di rotazione finale R sarà sempre identica, i valori numerici degli angoli restituiti $\mathbf{r}'$ potrebbero differire da quelli inseriti $\mathbf{r}$ in due casi specifici:

A. Intervallo di definizione (Rappresentazioni Equivalenti)

La funzione `arctan2(y, x)` restituisce valori nell'intervallo $[-\pi, \pi]$ (ovvero $[-180^\circ, 180^\circ]$). Inoltre, l'algoritmo di estrazione impone che l'angolo intermedio r_y si trovi sempre nell'intervallo $[-90^\circ, 90^\circ]$ definendo `sy` come una radice quadrata (sempre positiva).

- Se inserisci $r_y = 120^\circ$, la matrice generata sarà corretta, ma l'estrazione restituirà un set di angoli equivalenti (ad esempio $r'_y = 60^\circ$ con r_x e r_z sfasati di 180°).

- La rotazione fisica nello spazio è identica, ma la terna di angoli è diversa.

B. Gestione del Gimbal Lock ($r_y = \pm 90^\circ$)

Quando $r_y = 90^\circ$ o -90° , gli assi di r_x e r_z si allineano. Esistono infinite combinazioni di r_x e r_z che producono la stessa rotazione.

- Se inserisci $\mathbf{r} = [10^\circ, 90^\circ, 20^\circ]$, la funzione di estrazione imporrà arbitrariamente $r'_z = 0^\circ$ e restituirà $\mathbf{r}' = [30^\circ, 90^\circ, 0^\circ]$.
- Anche in questo caso, la matrice di rotazione associata a $[10^\circ, 90^\circ, 20^\circ]$ è numericamente identica a quella di $[30^\circ, 90^\circ, 0^\circ]$.

5. Script di Verifica e Validazione

Puoi utilizzare questo script Python per testare la complementarità delle tue funzioni sia in condizioni normali che in Gimbal Lock.

```
# --- TEST 1: Caso Standard ---
angoli_originali = np.array([15.0, 30.0, -45.0])
R1 = create_rot_matrix_zyx(angoli_originali)
angoli_estratti = rot_matrix_to_angles_zyx(R1)

print("=== TEST 1: CASO STANDARD ===")
print(f"Angoli Originali: {angoli_originali}")
print(f"Angoli Estratti: {angoli_estratti}")
print(f"Differenza: {np.abs(angoli_originali - angoli_estratti)}")
assert np.allclose(angoli_originali, angoli_estratti), "Errore nel caso standard!"

# --- TEST 2: Caso Gimbal Lock (Singularità ry = 90) ---
angoli_singolari = np.array([10.0, 90.0, 20.0])
R2 = create_rot_matrix_zyx(angoli_singolari)
angoli_estratti_singolari = rot_matrix_to_angles_zyx(R2)

print("\n=== TEST 2: GIMBAL LOCK ===")
print(f"Angoli Originali: {angoli_singolari}")
print(f"Angoli Estratti: {angoli_estratti_singolari} <- Nota: rz è forzato a")

# Verifichiamo se le due matrici di rotazione generate sono comunque identiche
R2_ricostruita = create_rot_matrix_zyx(angoli_estratti_singolari)
matrici_identiche = np.allclose(R2, R2_ricostruita)
print(f"Le matrici di rotazione coincidono? {matrici_identiche}")
assert matrici_identiche, "Errore: Le matrici non coincidono nel caso singolare!"
```